DX Backend Template

Volume 1 - Backend Template Guide

Node.js 22 | Express 5 | Sequelize 6 | Microsoft SQL Server

Bagian dari DNIA DX Development Starter Kit. Panduan ini membantu intern memahami tujuan template, struktur kode, cara menjalankan lingkungan lokal, aturan kontribusi, serta quality gate sebelum membuat perubahan.

> **Pesan utama**
> Gunakan template ini sebagai baseline. Ganti domain contoh Work Order sesuai kebutuhan proyek, tetapi pertahankan pemisahan layer, kontrol keamanan, pengujian, dan quality gate.

Audience: DX Intern / Junior Back-End Developer
Document owner: PED / Digital Transformation
Classification: Internal / Proprietary

1. Apa Itu DX Backend Template?

DX Backend Template adalah fondasi backend internal untuk pengembangan aplikasi berbasis Node.js 22, Express 5, Sequelize 6, dan Microsoft SQL Server. Domain contoh di dalam template adalah autentikasi dan CRUD Work Order.

Tujuan	Makna untuk Intern
Mempercepat project setup	Struktur backend, security baseline, logging, dan quality gate sudah tersedia.
Menstandarkan arsitektur	Request mengikuti jalur Route, validation/sanitization, controller, service, model, lalu SQL Server.
Security by default	Template memakai JWT, HttpOnly cookie, CSRF untuk browser, bcrypt, Helmet, CORS allowlist, rate limiting, strict schema, dan sanitization.
Menjaga kualitas	Lint, automated test, architecture check, npm audit, dan SonarQube menjadi kontrol sebelum handover.

Batas penggunaan

- Template berstatus private dan UNLICENSED.
- Source code, konfigurasi, credential, dan dokumentasi tidak boleh didistribusikan tanpa otorisasi tertulis.
- Library pihak ketiga tetap mengikuti lisensi masing-masing.

2. Teknologi Utama

Area	Teknologi / Paket	Kegunaan
Runtime	Node.js >=22.12.0 <23	Runtime yang diwajibkan oleh project.
Web framework	Express 5	Menyediakan HTTP server dan REST API.
Data access	Sequelize 6 + mssql/tedious	Mapping model dan akses Microsoft SQL Server.
Validation	Zod	Strict input schema sebelum controller.
Authentication	jsonwebtoken + bcrypt	JWT dan password hashing.
Security middleware	Helmet, CORS, express-rate-limit	Security headers, origin allowlist, dan request rate limit.
Logging	Winston + daily rotate	Structured JSON log, rotation harian, dan retention.

Area	Teknologi / Paket	Kegunaan
Export	ExcelJS + PDFKit	Membuat file Excel dan PDF.
Testing	node:test + Supertest	Automated test dan HTTP test.
Code quality	ESLint + SonarQube	Linting dan static analysis.

3. Gambaran Arsitektur

```
Client
  -> Route
  -> Middleware
  -> Validation / Sanitization
  -> req.validated
  -> Controller
  -> Service
  -> Model / Sequelize
  -> Microsoft SQL Server
```

Aturan inti
Controller menangani HTTP request/response. Service menangani business logic, transaksi, dan akses model. Model hanya memetakan tabel. Controller tidak boleh mengakses model secara langsung.

4. Struktur Folder dan Tanggung Jawab

Folder / File	Tanggung Jawab
src/config	Konfigurasi terpusat, misalnya database dan logger.
src/constants	Nilai domain immutable menggunakan Object.freeze().
src/controllers	Menerjemahkan HTTP request dan response.
src/errors	Error application yang konsisten.
src/integrations	Adapter layanan pihak ketiga. Gangguan integration tidak boleh memutus transaksi REST.
src/jobs	Pekerjaan terjadwal.
src/middleware	Authentication, authorization, CSRF, dan cross-cutting request controls.

Folder / File	Tanggung Jawab
src/models	Mapping tabel melalui Sequelize, bukan tempat mengubah schema.
src/routes	Definisi endpoint dan rangkaian middleware.
src/sanitizers	Pembersihan dan normalisasi input.
src/services	Business logic, transaksi, dan operasi model.
src/utils	Helper yang dapat digunakan kembali.
src/validations	Kontrak input berbasis schema.
src/app.js	Konfigurasi aplikasi Express.
src/server.js	Entry point untuk menjalankan server.
database	SQL script untuk lifecycle schema.
postman	Collection untuk mencoba dan memverifikasi API.
scripts	Runtime, architecture, database verification, dan seed automation.
test	Automated test untuk security, runtime, sanitization, dan quality tooling.

5. Quick Start Lokal

Ikuti urutan berikut. Jangan commit file .env atau credential lokal.

```
nvm use
npm ci
Copy-Item .env.example .env
```

Isi credential lokal pada .env. Buat schema menggunakan database/001_create_schema.sql. Aplikasi sengaja tidak menggunakan sequelize.sync().

```
npm run verify:database
$env:DEMO_ADMIN_PASSWORD = "ubah-dengan-password-kuat"
npm run seed
npm run dev
```

Health check tersedia pada GET /health. Password demo wajib diambil dari environment variable dan minimal 12 karakter.

Catatan keamanan

> Hapus atau ganti data demo sebelum production. Gunakan secret berbeda pada setiap environment dan lakukan rotasi jika ada indikasi kebocoran.

6. Contoh Domain dan Database

Template menyediakan dua tabel contoh dalam schema dbo.

Tabel	Fungsi	Kontrol Penting
APP_M_USER	Master user aplikasi	Username unik, password hash, role ADMIN/OPERATOR/VIEWER, active flag, timestamp UTC.
APP_T_WORK_ORDER	Transaksi Work Order	Nomor unik, quantity > 0, status terkontrol, created/updated by, soft-delete melalui active flag.

Schema dibuat dalam transaction, menggunakan foreign key ke APP_M_USER, dan memiliki index pencarian untuk active_flag, status, serta planned_date.

7. Authentication dan Authorization

Skenario	Mekanisme
Browser	JWT pada cookie HttpOnly dan signed double-submit CSRF. Browser tidak boleh menyimpan JWT di localStorage.
Postman / system-to-system	Bearer JWT dari POST /api/v1/auth/token. CSRF tidak diperlukan untuk Bearer.
Password	Hash menggunakan bcrypt.
Viewer	Read dan export.
Operator	Read, create, dan update.
Admin	Seluruh operasi.

8. API yang Tersedia di Postman

Area	Method	Endpoint
Health	GET	/health
Create bearer token	POST	/api/v1/auth/token

Area	Method	Endpoint
Current user	GET	/api/v1/auth/me
List Work Orders	GET	/api/v1/work-orders
Create Work Order	POST	/api/v1/work-orders
Update Work Order	PATCH	/api/v1/work-orders/{id}
Delete Work Order	DELETE	/api/v1/work-orders/{id}
Export Excel	GET	/api/v1/work-orders/export.xlsx
Export PDF	GET	/api/v1/work-orders/export.pdf

Postman collection menyimpan access token dari response ke collection variable bearerToken untuk dipakai pada request berikutnya.

9. Quality Gate dan Automated Controls

Perintah / Control	Apa yang Diverifikasi
npm run lint	ESLint dan aturan JavaScript project.
npm test	Automated tests dengan node:test.
npm run check:architecture	Menolak CommonJS, sequelize.sync(), raw query di src, dan akses model langsung dari controller.
npm run verify:database	Memastikan APP_M_USER dan APP_T_WORK_ORDER tersedia.
npm audit	Memeriksa kerentanan dependency.
npm run sonar	Menjalankan SonarQube menggunakan URL, token, dan project key dari environment.
npm run check	Menjalankan lint, test, dan architecture check.

Automated tests yang sudah disediakan

Test	Fokus
log-redaction.test.js	Authorization dan password disensor menjadi [REDACTED].
runtime-constants.test.js	Node 22.12+ dan constants yang frozen.
sanitization.test.js	Strict schema, unsafe sorting, pagination limit, control characters, formula Excel, dan filename safety.
security-middleware.test.js	Operator diizinkan pada role yang tepat dan

Test	Fokus
	Viewer menerima FORBIDDEN.
sonar.test.js	Environment wajib, URL valid, scanner tanpa shell, dan exit code diteruskan.

10. Workflow Kontribusi Intern

Tahap	Yang Harus Dilakukan
1. Pahami tiket	Baca requirement dan identifikasi route, schema, service, model, serta test yang terdampak.
2. Buat branch	Buat branch dari main.
3. Pertahankan layer	HTTP di controller, business logic di service, mapping data di model.
4. Validasi input	Semua input harus melalui schema dan sanitization menuju req.validated.
5. Tambahkan test	Tambahkan test untuk perubahan perilaku atau security.
6. Jalankan quality gate	Jalankan npm run check. Sebelum handover, jalankan npm audit dan SonarQube sesuai environment.
7. Buat pull request	Jelaskan perubahan, dampak API/database, bukti test, dan risiko.

11. Yang Boleh dan Tidak Boleh

Do	Do Not
Gunakan Node 22 sesuai .nvmrc dan engine project.	Jangan memakai runtime lain tanpa persetujuan.
Gunakan environment variable untuk credential.	Jangan commit .env, token, password, private key, atau export data.
Akses model melalui service.	Jangan import model langsung dari controller.
Ubah schema melalui SQL script yang direview.	Jangan memanggil sequelize.sync().
Gunakan ORM dan parameterized operation.	Jangan menambahkan raw SQL ke src.
Laporkan vulnerability secara privat.	Jangan membuat issue publik berisi exploit, credential, data produksi, atau detail infrastruktur.

12. Tugas Onboarding yang Disarankan

Checklist berikut adalah rekomendasi onboarding, bukan requirement eksplisit dari repository.

- [ ] Jalankan project sampai health check berhasil.
- [ ] Import Postman collection dan coba create token serta endpoint /auth/me.
- [ ] Pelajari alur List Work Orders dari route ke model.
- [ ] Tambahkan satu field non-kritis pada domain contoh melalui perubahan schema terkontrol, validation, model mapping, service, dan test.
- [ ] Jalankan npm run check dan perbaiki seluruh error.
- [ ] Jelaskan kembali mengapa controller tidak boleh mengakses model secara langsung.

13. Kapan Harus Bertanya kepada Mentor?

- Perubahan membutuhkan tabel, constraint, index, atau migration baru.
- Perubahan menyentuh authentication, authorization, CSRF, cookie, CORS, atau security headers.
- Perubahan membutuhkan integration eksternal atau data produksi.
- Requirement mendorong penggunaan raw SQL, penyimpanan token di browser, atau penghapusan quality gate.
- npm audit atau SonarQube menemukan issue yang belum dipahami.

14. Referensi Repository

Dokumen ini disusun dari file template backend berikut:

- README.md
- CONTRIBUTING.md
- SECURITY.md
- NOTICE.md
- package.json
- eslint.config.js
- run-sonar.js
- scripts/check-runtime.js
- scripts/check-architecture.js
- scripts/verify-database-schema.js
- scripts/seed-demo.js
- database/001_create_schema.sql
- postman/DX-Template.postman_collection.json
- test/log-redaction.test.js
- test/runtime-constants.test.js
- test/sanitization.test.js
- test/security-middleware.test.js
- test/sonar.test.js

> **Sebelum mulai coding**
> Baca README.md, CONTRIBUTING.md, SECURITY.md, lalu jalankan project secara lokal. Jangan melewati quality gate hanya untuk mempercepat penyelesaian.